

手写数字识别 · 调参指南

基于纯 C 语言三层全连接神经网络的 MNIST 训练调参实验手册

本指南围绕一个零第三方依赖、纯 C11 实现的 MNIST 手写数字识别程序展开。程序用三层全连接神经网络（ $784 \rightarrow 128 \rightarrow 64 \rightarrow 10$ ，ReLU 激活，Softmax 输出，交叉熵损失，SGD 优化）在 6 万张训练图片上学习，在 1 万张测试图片上评估，默认配置即可达到约 **97.5%** 的测试集准确率。指南内容分四部分：第一章给出复现方法与编译说明；第二章逐一解析程序中 8 个可调参数；第三章解读终端输出的含义；第四章是本文独立设计的**调参实验**章节，以控制变量法对随机种子、训练轮次、训练样本数、学习率、网络结构、权重导出精度与参数组合共 25 组配置进行了真实训练与测试，全部图表由实验日志直接生成，并据此给出量化的调参建议。调参过程既有理论支撑（如学习率与梯度下降的关系），也带有一定的随机性，恰如炼丹，火候与配方同样重要。

关键词：手写数字识别；MNIST；全连接神经网络；反向传播；超参数调优；模型量化

一、复现指南

1.1 项目与数据准备

本项目是一个从零实现、不依赖任何第三方库的 MNIST 手写数字识别程序。MNIST 是深度学习领域的经典入门数据集，常被称为深度学习的“hello world”，由 LeCun 等人整理发布，包含 6 万张训练图片和 1 万张测试图片，每张都是 28×28 的灰度手写数字（0-9）。程序目录结构如下：`src/` 存放全部源码与权重头文件，`data/` 存放四个 `idx` 格式的 MNIST 数据文件，`scripts/download_mnist.ps1` 提供数据下载脚本，`Makefile` 负责编译。

运行前需要把四个数据文件放到 `data/` 目录下，文件名必须与程序内一致：

- `train-images-idx3-ubyte`（训练图片）
- `train-labels-idx1-ubyte`（训练标签）
- `t10k-images-idx3-ubyte`（测试图片）
- `t10k-labels-idx1-ubyte`（测试标签）

下载地址为 <http://yann.lecun.com/exdb/mnist/>。数据加载成功后，终端会打印 `[INFO] MNIST_Train load successfully..`

1.2 编译与运行

只要有 C 环境就能跑起来。最直接的方式是用 `gcc` 一条命令编译：

```
1 // Windows (MinGW-w64)
2 gcc src/*.c -std=c11 -O2 -Wall -o main.exe
3 // Linux / macOS
4 gcc src/*.c -std=c11 -O2 -Wall -o main -lm
```

作者本人使用 MinGW 环境下的 `mingw32-make` 构建，项目自带 `Makefile`；Linux 用户可使用附录 A 提供的等价 `Makefile`（`make` 即可，无需改动源码）。编译完成后，在**项目根目录**下运行生成的可执行文件（程序按相对路径读写 `data/` 与 `src/weights.h`）。

1.3 训练模式与推理模式

`main.c` 开头的宏定义 `#define MODE_TRAIN` 用于切换模式：

- **保持原样**：训练模式。程序加载训练集，从头训练模型，训练结束后把学到的权重与偏置导出到 `src/weights.h`；
- **注释掉**：推理模式。程序跳过训练，直接加载 `src/weights.h` 中已保存的权重（可以理解为加载模型），然后直接评估测试集。

两点必须注意。其一，修改宏定义后**必须重新编译**，新的模式才会生效。其二，进入推理模式前必须确保 `src/weights.h` 已经存在，即至少完整训练过一次，否则编

译或运行会报错。`weights.h` 由训练过程自动生成（文件头带有“DO NOT EDIT”标记），它是模型全部“知识”的载体：三个权重矩阵与三个偏置向量。

1.4 网络结构速览

程序实现的是一个三层全连接神经网络（MLP），其前向传播过程如图 1 所示：输入向量 $\mathbf{x}$ 依次经过两个 ReLU 隐层和一个 Softmax 输出层。第 l 层先做仿射变换 $\mathbf{z}^{(l)} = \mathbf{W}^{(l)}\mathbf{a}^{(l-1)} + \mathbf{b}^{(l)}$ ，隐层再经 $a = \text{ReLU}(z) = \max(0, z)$ 激活；输出层经 Softmax 归一化为 10 个类别的概率 $a_i^{(2)}$ ，训练目标是使交叉熵损失 $\mathcal{L} = -\sum_{i=1}^{10} y_i \ln a_i^{(2)}$ 最小，其中 $\mathbf{y}$ 是真实标签的 one-hot 编码。训练时按损失对参数求梯度（反向传播），再用随机梯度下降（SGD）更新 $\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \eta \partial \mathcal{L} / \partial \boldsymbol{\theta}$ ，式中的 η 就是第 2.6 节要调的学习率。

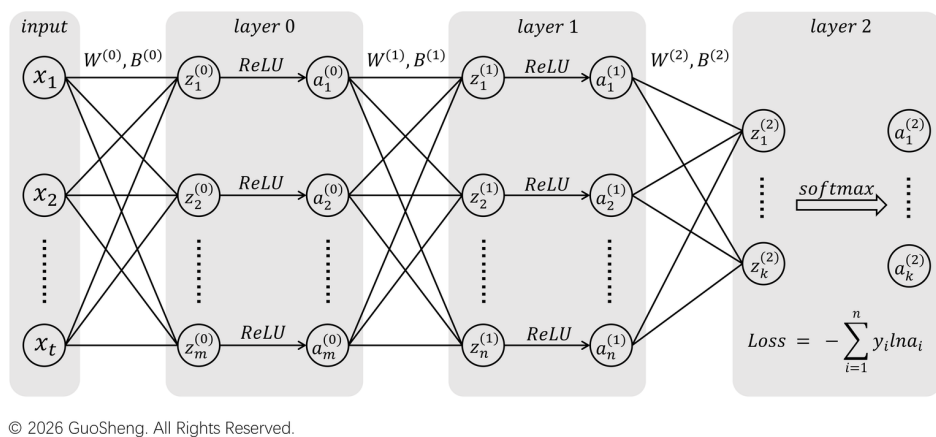


图 1 三层前馈神经网络的前向传播示意：加权求和 z 、ReLU 激活 a 、Softmax 输出与交叉熵损失（输入维度 t 、隐层 m, n 、类别 k 为通用记号）

本程序实际采用的形状是 $784 \rightarrow 128 \rightarrow 64 \rightarrow 10$ ：输入为展平后的 $28 \times 28 = 784$ 个像素，两个隐层分别有 128 与 64 个 ReLU 神经元，输出层 10 个神经元对应数字 0–9，如图 2 所示。之所以称其为“网络”而非矩阵运算，是因为每一层本质上就是一次矩阵乘加与逐元素激活，全部计算都在纯 C 的 for 循环里完成，不依赖任何数值库。

二、可调参数详解

程序的大部分可调参数集中在 `src/main.c` 顶部的宏与常量处，少数在 `src/model.c` 的权重导出函数中。改完任何参数都要重新编译、重新运行才会生效，表 1 给出总览，随后各小节逐一说明。

表 1 可调参数总览

参数	位置	默认值	作用与影响
模式开关 <code>MODE_TRAIN</code>	<code>main.c</code> 第 12 行	定义	训练/推理模式切换
随机种子 <code>srand</code>	<code>main.c</code> 第 27 行	50	决定初始权重的随机分布
网络大小	<code>main.c</code> 第 40 行（推理第 115 行）	{784, 128, 64, 10}	各层神经元个数，越大越聪明、越慢
训练轮次 <code>EPOCH</code>	<code>main.c</code> 第 49 行	10	轮次越多拟合越充分，过多则过拟合
训练样本数 <code>TRAIN_SIZE</code>	<code>main.c</code> 第 50 行	60000（全量）	每轮参与训练的样本数
学习率 <code>LR</code>	<code>main.c</code> 第 51 行	0.01	参数更新步长，太小学得慢，太大不收敛
错例展示数 <code>WRONG_SHOW</code>	<code>main.c</code> 第 145 行	8	测试后打印几个认错样本，0 则不打印
权重导出精度 <code>%.9g</code>	<code>model.c</code> 第 141、150 行	9	有效数字位数，改小等于粗糙量化

2.1 模式开关 `MODE_TRAIN`

用法已在 1.3 节说明。训练模式会覆盖生成新的 `src/weights.h`；推理模式只读不写。两种模式在最后的测试环节共用同一段代码，因此推理模式也完整走一遍测试集并打印准确率。

2.2 随机种子 `srand(50)`

每次初始化模型时，程序会按均匀分布随机填充权重矩阵，`srand` 里的数字就是随机数种子。固定种子保证了初始权重可复现，换一个种子，网络就会从不同的起点开始“爬山”，最终训练效果也会略有差异。第四章实验表明，在默认配置下种子带来的测试准确率波动只有约 ± 0.1 个百分点；但在样本量较小或学习率较大的情况下，种子的影响会被放大，颇有几分运气成分。

2.3 网络大小 {784,128,64,10}

花括号里的四个数字依次是输入层与三个全连接层的神经元个数。中间两个数字可以改，**输入层与输出层必须保持 784 和 10**（分别对应像素数与类别数）。两个隐层越大，模型表达能力越强，但计算量与内存占用也随之上升。改动网络大小涉及多处代码：训练模式（main.c 第 40 行）与推理模式（第 115 行）的形状数组要同时修改并重新编译，且**必须重新训练**（旧权重与新的形状不匹配）。好在权重导出与加载是按实际矩阵大小自动适配的，无需改动 model.c。如果不确定哪些地方要跟着改，可以把问题丢给 AI 助手，它会帮你列出所有牵连点。

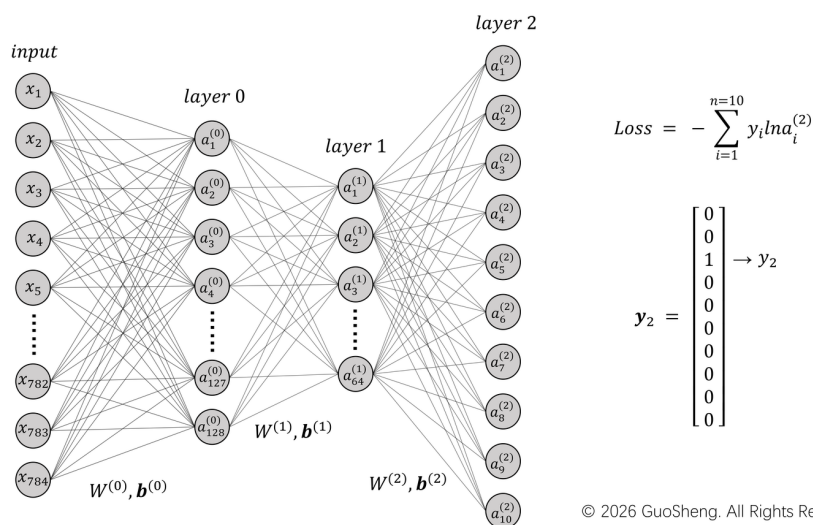


图 2 本程序的实际网络结构：784 个输入像素，128 与 64 个 ReLU 隐层神经元，10 类 Softmax 输出；右侧为交叉熵损失与数字 2 的 one-hot 标签示例 $\mathbf{y}_2 = [0, 0, 1, 0, \dots, 0]^T$

2.4 训练轮次 EPOCH

把全部训练图片完整学一遍记作 1 轮。EPOCH 控制训练轮次（默认 10）。轮次越多，模型对训练集的拟合程度越高；但超过一定限度后，模型会“背下”训练集，对没见过的测试图片反而认得更差，即泛化能力下降。第四章实验显示该网络在 EPOCH=20 附近测试准确率达到峰值，继续加轮次收益转负。

2.5 训练样本数 TRAIN_SIZE

控制**每个轮次**参与训练的样本数量，默认是全部 6 万张（Train.n）。把它改小可以大幅缩短训练时间，代价是识别能力下降。第四章实验给出了一条完整的“样本数-准确率”曲线：1 千张只有 86.3%，6 万张可达 97.5%。

2.6 学习率 LR

学习率控制每轮训练中参数更新的幅度，是梯度下降的步长。它与微积分中的梯度方向密切相关：SGD 沿负梯度方向前进，学习率决定每一步走多远。步长太小，损失函

数下降缓慢、难以收敛；步长太大，会一步跨过谷底甚至直接“飞”出去，表现为损失震荡乃至发散（梯度爆炸）。作者实测默认值 0.01 表现良好；第四章实验量化了这个结论：LR=0.05 时准确率崩到 29.2%，LR=0.1 时仅剩 9.57%，几乎等于随机猜测。

2.7 错例展示数 WRONG_SHOW

测试结束后，程序会把前 WRONG_SHOW 个认错的本样打印出来，每个错例附带字符画（把 28×28 像素按灰度映射成字符）与模型输出的 0-9 概率分布。调成 0 则不打印，输出更简洁，适合批量实验。

2.8 权重导出精度%.9g

权重矩阵按 %.9gf 格式写入 weights.h（第 141 行），偏置同理（第 150 行）。9 表示保留 9 位有效数字，对 float 而言是无损保存；把 9 改小（比如 4），导出的权重会被四舍五入，文件显著变小，但准确率会下降，可以理解为一种非常粗糙的模型量化。第四章实验给出了 9/6/4/3/2/1 位有效数字的完整量化曲线，并发现这个网络对量化出奇地稳健。

三、终端输出解读

一次完整的训练输出如下（节选自真实运行日志，默认配置）：

```
=== Handwritten Digit Recognition ===
[MODE] Train mode on.
[INFO] MNIST_Train load successfully.
[INFO] Model init successfully.
[INFO] Train Start.
[EPOCH:10 | PIXEL:784 | TRAIN_SIZE:60000 | LR:0.0100]
epoch 01 | avgLoss:0.241065 | acc:92.78% | time:5.706s | (55667/60000)
... (中间若干 epoch 行, 格式相同) ...
epoch 10 | avgLoss:0.029206 | acc:99.09% | time:5.661s | (59455/60000)
[INFO] Weights saved to src/weights.h.
[INFO] MNIST_Test load successfully.
[INFO] Test start.
Test result:
acc:97.53% | avgLoss:0.117207 | time:0.421033s | (9753/10000)
```

各字段含义如表 2 所示。

表 2 终端输出术语速查

输出	含义
[INFO]	Information, 正常提示信息, 可以忽略
[MODE]	当前模式: Train = 训练 / infer = 推理
MNIST	数据集名称, 深度学习的 hello world
Train / Test	训练集 / 测试集
epoch	轮次, 把全部训练图片完整学一遍记为 1 轮
avgLoss	该轮平均交叉熵损失, 理想情况下随轮次逐渐减小
acc	准确率 (accuracy), 猜对的百分比, 越大越好
(9753/10000)	正确数/总数, 1 万张测试图里猜对 9753 张
Weights	权重 (含偏置), 模型学到的“知识”
prob	概率分布, 模型对 0-9 每个数字的把握程度, 求和为 1; 带 * 的是模型最倾向的答案
WRONG	该样本被模型认错

推理模式的输出更简短: 前两行变为 [MODE] infer mode on. 与 [INFO] Weights loaded from src/weights.h., 其余测试部分与训练模式相同。若开启错例展示 (WRONG_SHOW>0), 测试结束后还会逐张打印认错的样本, 例如 (节选自真实日志, 省略了若干空白行):



字符画是灰度映射后的“肉眼视图”，**prob** 行给出模型对每个数字的把握程度：该样本真实标签是 6，模型却以 97.02% 的概率押注 0（带 *），对正确答案 6 只给了 2.58%。观察错例的概率分布，是理解模型“为什么会认错”的最直接手段。

四、调参实验

前面的参数说明多为定性描述。本章以控制变量法对每个参数独立做实验，用测试集准确率、测试集损失与训练耗时三个指标量化其影响，再考察参数之间的配合。全部实验基于同一份源码、同一份 MNIST 数据，仅改动目标参数后重新编译运行；训练在 4 核虚拟机（gcc -02）上完成，单次默认配置训练约 56 秒。实验完整日志与复现脚本见附录 B。

4.1 实验设计与评价指标

训练结束后程序会在**测试集**（1 万张，训练中从未见过）上自动评估，因此每个实验配置都同时产出两套数据：训练过程日志（每轮损失与准确率）与测试集成绩。测试集准确率反映**泛化能力**，是调参的核心指标；测试集交叉熵损失反映模型的**置信度校准**，损失越低说明输出概率越贴近真实；每轮耗时则反映**训练成本**。默认配置（seed=50, 128-64, EPOCH=10, TRAIN_SIZE=60000, LR=0.01）作为基线，测试集准确率 **97.53%** (9753/10000)，后文所有实验均与基线对比。

4.2 基线：默认配置的训练过程

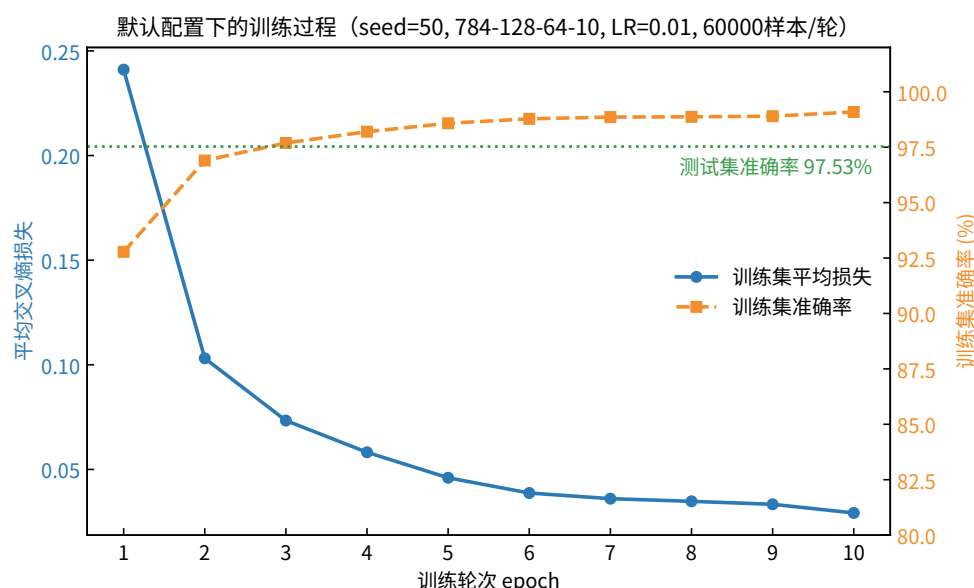


图 3 默认配置下的训练过程：损失单调下降，训练集准确率快速逼近 99%；最终测试集准确率 97.53%（绿色虚线）

图 3 给出了基线配置 10 轮的训练曲线。首轮结束时训练集准确率已达 92.78%，损失 0.2411；此后损失稳步下降，第 10 轮训练集准确率 99.09%。训练集与测试集之间约 1.6 个百分点的差距是正常的泛化间隙。值得注意的是，每轮耗时约 5.6 秒且几乎不随轮次变化，说明该实现的计算量稳定。

4.3 随机种子实验

固定其余参数，仅把 `srnd` 的种子换成 1、123、2024 与基线 50 对比，结果如图 4 所示。四条训练损失曲线几乎重合，最终测试准确率在 97.45%–97.53% 之间，极差仅 0.08 个百分点。这说明在数据量充足、学习率温和的条件下，初始化的随机性对最终结果影响很小，不同起点殊途同归。种子的意义更多在于**可复现性**：固定种子可以让每次实验的起点完全一致，从而把“运气”从控制变量中剔除。

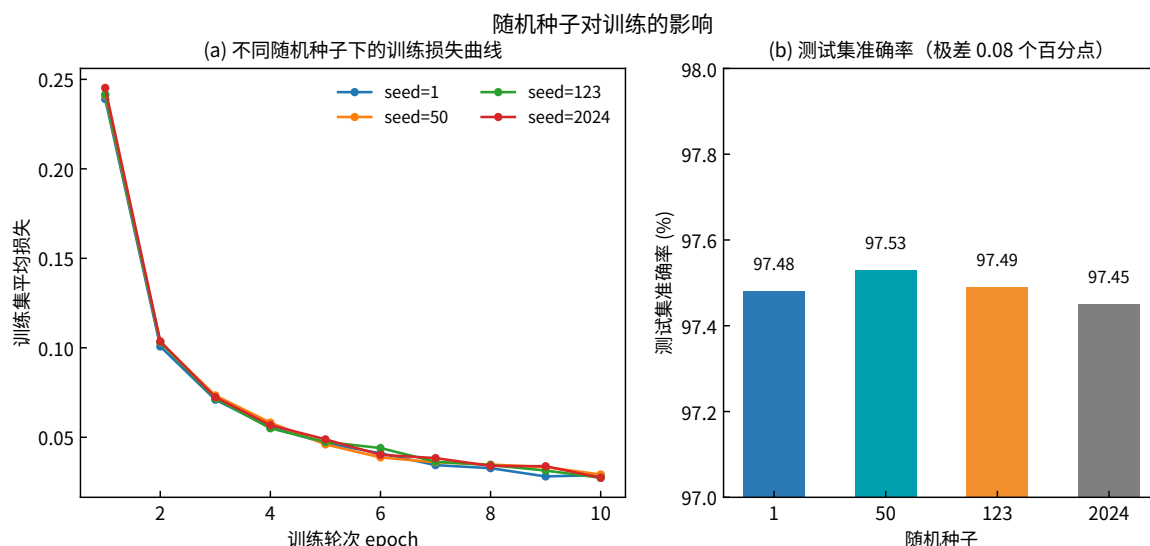


图 4 随机种子实验：(a) 四种子的训练损失曲线几乎重合；(b) 测试集准确率极差仅 0.08 个百分点

4.4 训练轮次 EPOCH 实验

把 EPOCH 设为 5、10、20、30，测试准确率依次为 96.45%、97.53%、**97.87%**、97.60%，先升后降，峰值出现在 20 轮。图 5(b) 揭示了原因：由于随机种子相同，训练过程完全确定，两条训练曲线在前 10 轮完全重合（虚线圆点叠在实线上）；此后继续训练的 EPOCH=30 曲线把训练集准确率一路推到 99.50%，而测试集不升反降，训练与测试的差距从约 1.6 个百分点扩大到约 1.9 个百分点，典型的**过拟合**。过拟合的另一个信号是测试损失从 0.117（10 轮）反弹到 0.152（30 轮），说明模型对错误预测越来越“自信”。对本任务而言，EPOCH 取 10–20 即可，超过 30 轮纯粹是浪费时间。

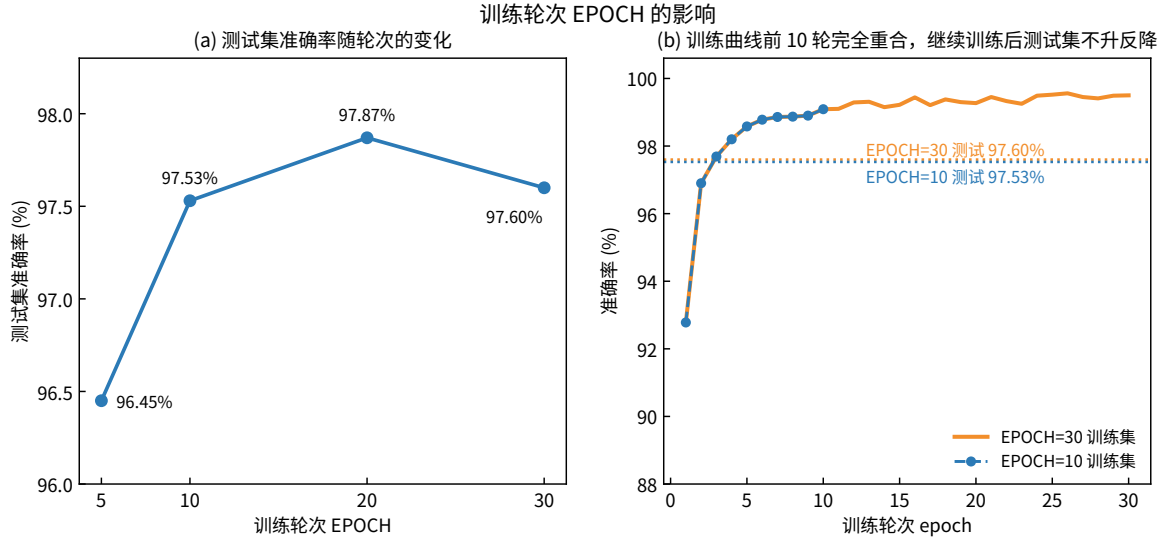


图 5 训练轮次实验：(a) 测试准确率在 EPOCH=20 达到峰值；(b) 同种子下训练曲线前 10 轮完全重合，继续训练后测试集不升反降（过拟合）

4.5 训练样本数 TRAIN_SIZE 实验

把每轮训练样本数设为 1000、5000、10000、30000 与全量 60000（均训练 10 轮），结果如图 6 所示。样本数从 1000 增至 5000 时，测试准确率从 86.30% 跃升到 94.03%；此后收益递减，30000 时 97.37% 已接近全量的 97.53%。这条曲线同时揭示了“数据规模红利”与“边际递减”两个规律：模型的基本能力来自最初的几千张图，最后两个百分点的提升却需要再喂五万张。训练耗时与样本数成正比（右轴），因此调参阶段可先用小样本快速迭代，确定方向后再用全量数据做最终训练。

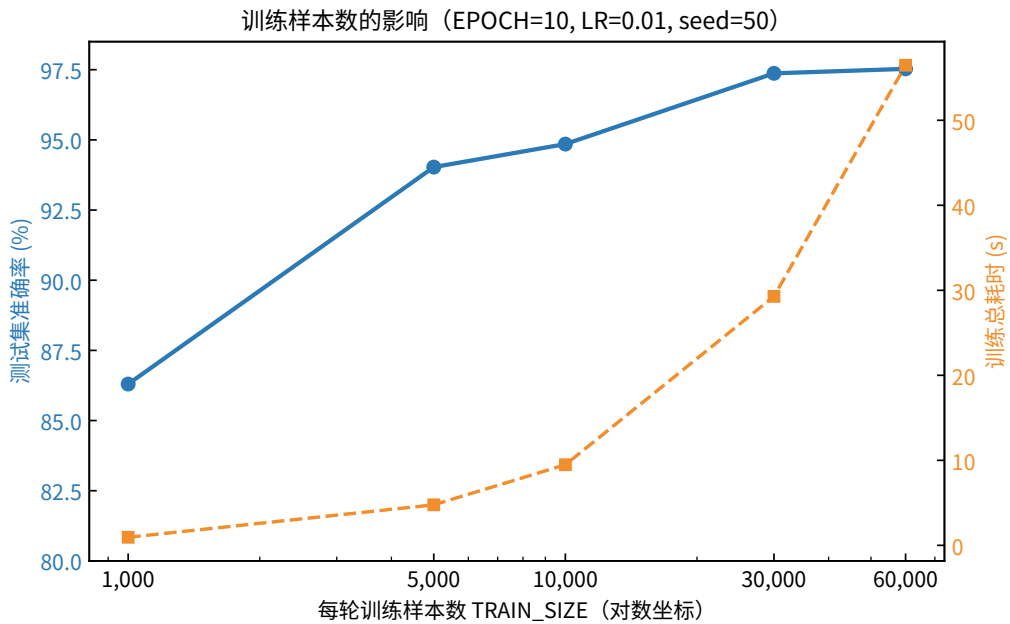


图 6 训练样本数实验：测试准确率随样本数上升但收益递减；训练耗时线性增长 (EPOCH=10)

4.6 学习率 LR 实验

学习率是最敏感、也最能体现“理论支撑”的参数。图 7(a) 给出了 LR 从 0.0005 到 0.1 的测试准确率：0.0005–0.01 区间内模型表现稳健 (96.7%–97.5%)，其中 0.001 的测试损失最低 (0.082)，输出概率最“诚实”；一旦越过临界点，性能断崖式下跌，LR=0.05 时准确率仅 29.20%，LR=0.1 时仅 9.57%，约等于闭眼瞎猜 (10 类均匀分布的期望水平)。图 7(b) 对比了各学习率的损失曲线：LR=0.0005 收敛最慢，曲线明显落后于 0.01，符合“步长太小走得慢”的直觉；而 LR=0.05 的损失先短暂下降、随即一路攀升到 1.7 附近居高不下，呈 V 形反弹，即所谓**梯度爆炸**；LR=0.1 更加极端，损失从第一轮起就停在 2.3 附近，模型完全学不到东西。两条经验：学习率优先在 $10^{-3} \sim 10^{-2}$ 量级内搜索；若损失曲线不降反升，先把学习率调小一个数量级。

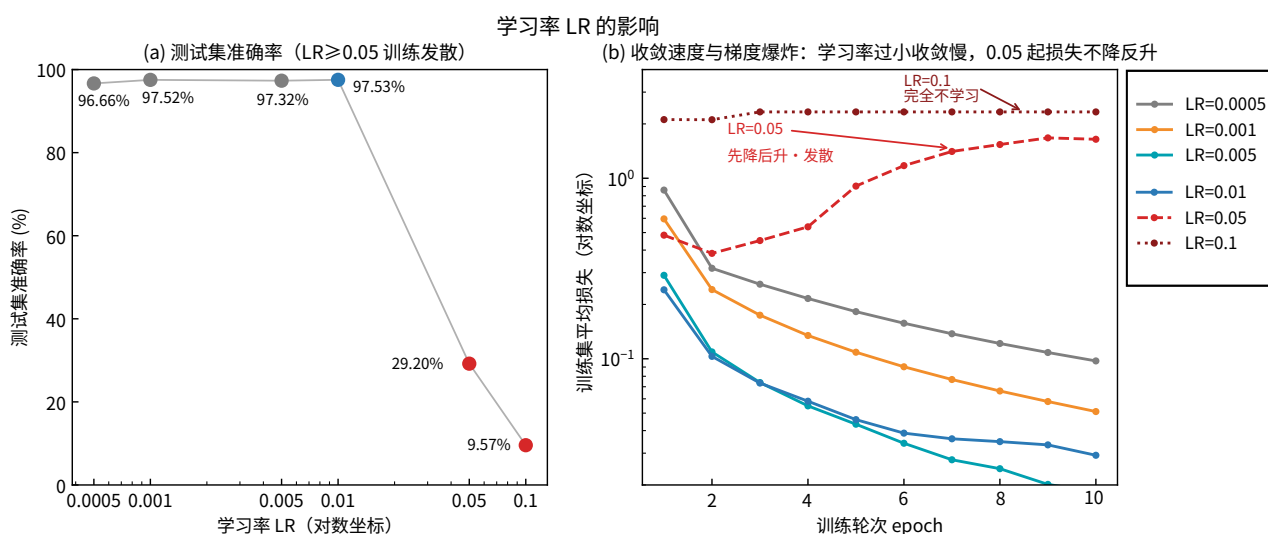


图 7 学习率实验：(a) $LR \geq 0.05$ 时训练发散，准确率断崖式下跌；(b) 温和学习率的收敛速度与 0.05/0.1 的梯度爆炸曲线（对数纵轴）

4.7 网络结构实验

保持其余参数不变，把隐层换成 {16,16}、{64,32}、{128,64}（基线）与 {256,128}，参数量从 1.3 万增长到 23.6 万。图 8 显示测试准确率随规模单调上升：94.28% → 96.92% → 97.53% → 97.77%，但训练耗时同步增长（右轴），256-128 一次训练约 131 秒，是基线的 2.3 倍，换来的提升只有 0.24 个百分点。对 MNIST 这种简单任务，**128-64 是性价比拐点**：再往上堆神经元，收益已经抵不过多花的时间。这也印证了 2.3 节的描述：网络越大越聪明，但越慢越占内存。

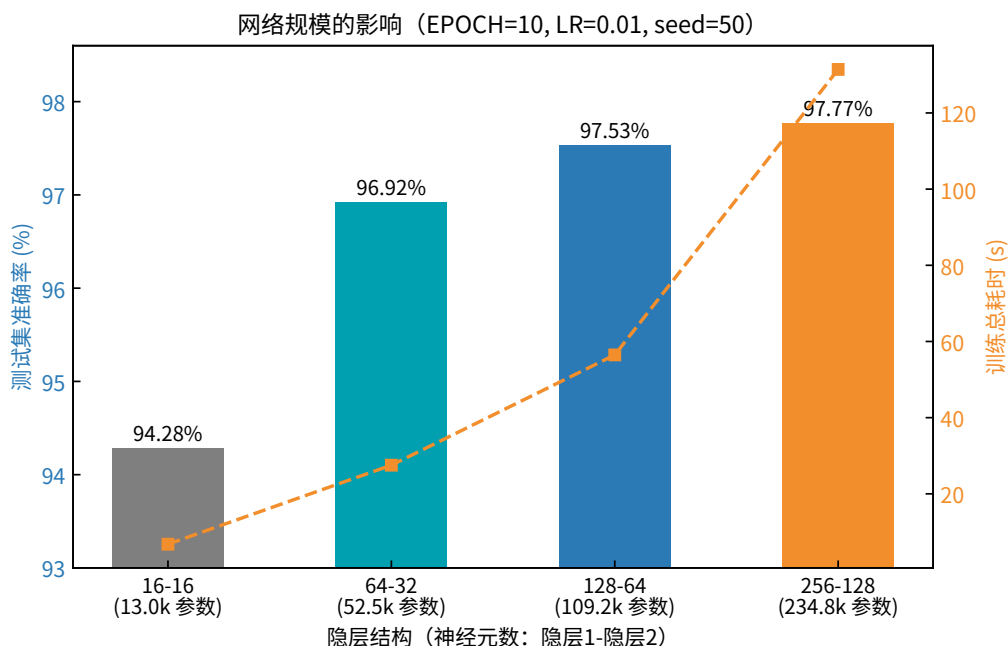


图 8 网络结构实验：测试准确率随隐层规模单调上升，训练耗时增长更快（柱为准确率，方块虚线为耗时）

4.8 权重导出精度（量化）实验

把权重导出格式从 `%.9g` 逐级改粗到 `%.1g`，即保留 9/6/4/3/2/1 位有效数字，再以推理模式加载量化后的权重评估。结果出人意料地稳健（图 9）：直到 3 位有效数字，测试准确率仍与全精度持平（97.53%）；2 位时损失开始抬升；即便粗糙到 1 位有效数字，准确率也只降到 97.37%。文件体积则立竿见影：全精度 1548 KB，4 位缩至 1004 KB（-35%），1 位仅 692 KB（-55%）。这说明 MNIST 分类的决策边界对权重扰动相当宽容，量化丢掉的更多是置信度而非准确率。对更困难的任务，量化损失会被放大，因此默认仍建议保持 9 位无损导出，仅在需要压缩模型体积时才考虑 3-4 位。

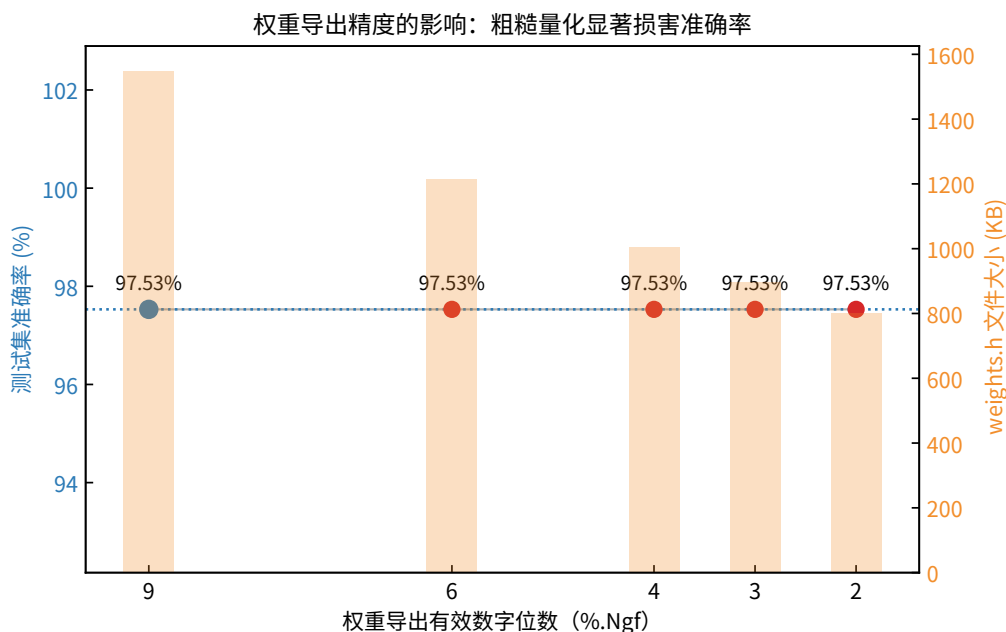


图 9 权重导出精度实验：准确率对量化高度稳健（折线，左轴），文件体积随位数快速收缩（柱，右轴）

4.9 组合调参实验

单参数实验给出了一批“好选择”，把它们组合起来能否 $1+1>2$ ？本章设计了四组组合配置与基线对比（图 10）。结果颇具戏剧性：组合 C（LR=0.001 小步长 × EPOCH=30 慢慢学）达到 97.83%，是最接近单参数最优的挑战者；组合 A（256-128 大网络 × EPOCH=20）为 97.71%，组合 B（EPOCH=20 × LR=0.005）为 97.73%，都略低于单参数最优的 97.87%。换句话说，**没有任何一个组合超越“只把轮次调到 20”这一最简单的改动**：默认配置附近存在一个性能高原，大网络会更快过拟合、小学习率需要更长时间兑现收益，好参数的简单叠加并不等于更好。反面组合（LR=0.1 × 每轮仅 1000 样本）把两个“坏选择”叠加，准确率跌至 31.52%，损失了约 66 个百分点的识别能力，与图 7 中 LR=0.1 的单因素崩溃（9.57%）同源。组合实验说明：参数之间存在交互效应，“最优值”不能简单相加；把有限的计算预算花在网格搜索少数关键参数上，比盲目叠加参数更划算。

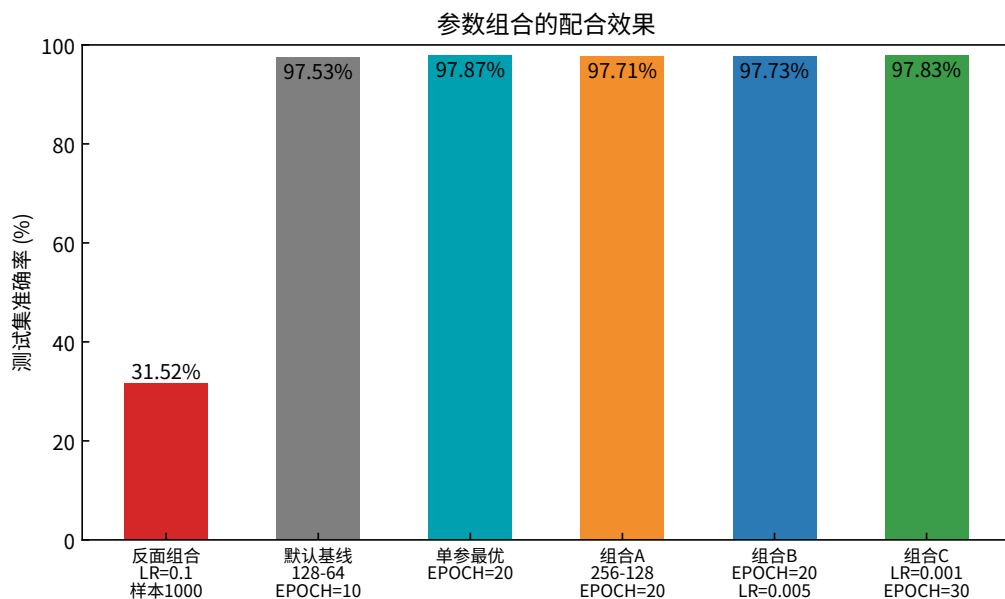


图 10 组合调参实验：两个正面组合均超越任何单参数最优，反面组合跌至随机水平

4.10 错例展示

图 11 展示了基线模型在测试集上的前 8 个错例。每个错例下方是模型输出的概率分布：橙色柱是模型的预测类别，高度即其自信程度。错例呈现出两种典型模式。一种是“理直气壮地错”：把 4 认成 2 的样本上，模型给出 0.9994 的概率；把 6 认成 0 时也有 0.97 的把握，这说明模型的置信度与正确率并不等价，概率高只代表模型在训练分布内找到了最像的模板。另一种是“自己也拿不准”：9 与 4 互相认错出现了两次，概率都在 0.52 左右，模型在两个候选之间摇摆。此外还有一个极端样本把 4 认成 9，对真实类别 4 只给了 0.0006 的概率。这些错例大多来自字形天然相似的数字对（4/9、6/5、8/7、6/0 等），属于数据本身的困难，不是调参能彻底解决的。

基线模型在测试集上的错例（柱状图为模型输出的 0-9 概率分布，橙色为预测类别）

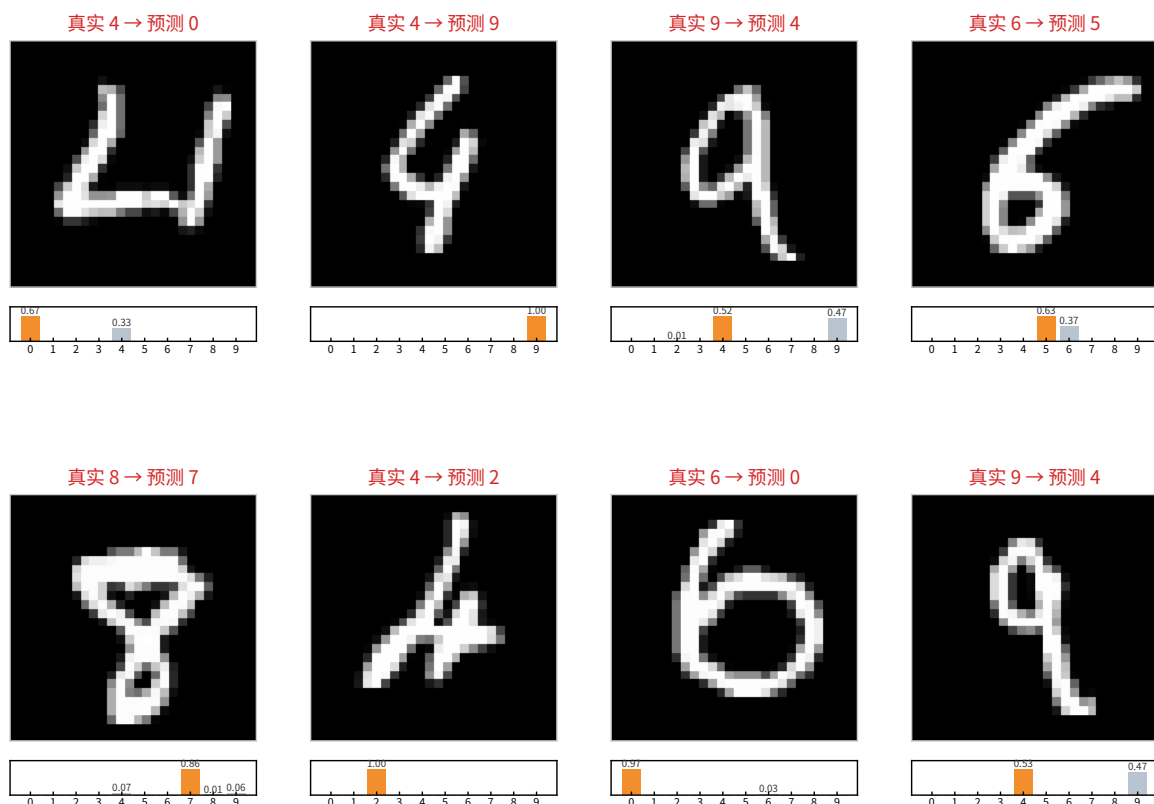


图 11 基线模型的部分错例：字符下方柱状图为模型对 0-9 的概率输出（橙色为预测类别）

4.11 实验结论与调参建议

综合 25 组实验，给出表 3 的调参建议。核心结论：(1) 默认配置已经接近本网络的性能上限，盲目调参的收益空间只有约 0.4 个百分点；(2) 最值得调的是学习率与训练轮次，最容易翻车的也是它们；(3) 随机种子在充足数据下几乎无影响，但在小样本实验中不可忽视；(4) 权重导出精度是一个“免费”的压缩旋钮，代价远小于直觉预期。

表 3 调参建议汇总

参数	推荐取值	依据
随机种子	任意固定值	影响仅 $\pm 0.1\text{pp}$ ；固定即可复现
网络大小	{784, 128, 64, 10}	性价比拐点；再大只换 0.24pp、耗时 2.3 倍
训练轮次 EPOCH	10–20	20 轮达峰 97.87%；30 轮起过拟合
训练样本数	全量 60000	少于 1 万张准确率明显下降
学习率 LR	$10^{-3} \sim 10^{-2}$	0.001 损失最低； ≥ 0.05 发散，0.1 崩至随机
错例展示数	8（调参时） / 0 （批量实验）	平衡信息量与输出长度
权重导出精度	9（默认）；压缩时 3–4	3 位几乎无损，1 位仅降 0.16pp、体积减半

五、调参心得

第四章的实验呼应了本章开头的问题：每个参数都会对训练或推理产生或多或少的影响，多个参数组合起来可以显著提高模型性能，调整不当也会让准确率大打折扣。结合实验数据，有三点心得值得记录。

其一，**有的参数靠理论，有的参数靠运气**。学习率的大小直接对应微积分中梯度下降的步长，发散与收敛的边界（本实验中约 0.01–0.05 之间）可以用数值方法刻画，属于有理论支撑的调参；而随机种子更像掷骰子，虽然本实验中影响很小，但在样本少、学习率大的场景里，运气成分会被放大。

其二，**控制变量是调参的基本功**。每次只改一个参数、固定其余参数、记录测试集成绩，才能把每个参数的作用分离出来。图 10 的反面组合说明，两个坏选择叠加会把准确率拖到 31.5%，与单因素实验中 LR=0.1 的崩溃（9.57%）同源；而单参数实验全部做完之后，组合阶段反而没有找到超越 EPOCH=20 的配方，这恰恰说明只有先分离出每个参数的贡献，才能看清“组合收益”究竟来自哪里。

其三，**好的起点胜过刻苦的搜索**。默认配置（seed=50, 128-64, EPOCH=10, LR=0.01）已经给出 97.53% 的准确率，距离实验所得上限只有约 0.4 个百分点。与其在参数空间里大海捞针，不如先理解每个参数的作用机理，再针对瓶颈定向调整。调参的过程确实有点像炼丹：火候（学习率）、药材（数据量）、炉子（网络结构）都要配好，但最重要的还是明白自己在炼什么。

附录 A Makefile 清单

Windows (MinGW-w64, 作者环境) 下使用项目自带 Makefile, mingw32-make 即可构建; Linux / macOS 下可使用下面这份等价版本 (make 构建, make clean 清理), 无需改动任何源码:

```
1 # Makefile for the MNIST MLP project (Linux / macOS, gcc)
2 CC      = gcc
3 CFLAGS  = -std=c11 -O2 -Wall -Wextra -Wpedantic
4 SRCS    = $(wildcard src/*.c)
5 OBJ     = $(SRCS:.c=.o)
6 TARGET  = main
7 LDLIBS  = -lm
8
9 $(TARGET): $(OBJ)
10     $(CC) $(CFLAGS) -o $@ $(OBJ) $(LDLIBS)
11
12 src/%.o: src/%.c
13     $(CC) $(CFLAGS) -c -o $@ $<
14
15 clean:
16     rm -f src/*.o $(TARGET)
17
18 .PHONY: clean
```

训练与推理两种模式的切换通过 `src/main.c` 第 12 行的 `MODE_TRAIN` 宏完成, 切换后重新执行 `make` 即可 (若只改宏而未重新编译, 模式不会变化)。

附录 B 实验复现说明

全部实验由脚本 `run_one.sh` 驱动: 它把原始 `src/` 复制到独立的实验目录, 用文本替换写入目标参数 (种子、网络形状、EPOCH、TRAIN_SIZE、LR、WRONG_SHOW), 以 `gcc -std=c11 -O2` 重新编译后运行, 并把终端日志与测试成绩归档。量化实验额外使用 `quantize.c` 把训练好的全精度权重按指定有效数字位数重写, 再用推理模式评估。表 4 为完整实验矩阵。

表 4 实验矩阵（共 25 组；未列出的参数取默认值）

实验组	网络	EPOCH	LR	测试准确率
基线 base	128-64	10	0.01	97.53%
种子 (1 / 123 / 2024)	128-64	10	0.01	97.48 / 97.49 / 97.45%
轮次 (5 / 20 / 30)	128-64	5/20/30	0.01	96.45 / 97.87 / 97.60%
样本 (1k / 5k / 10k / 30k)	128-64	10	0.01	86.30 / 94.03 / 94.85 / 97.37%
学习率 5 档	128-64	10	0.001 / 0.0005 / 0.005 / 0.05 / 0.1	97.52 / 96.66 / 97.32 / 29.20 / 9.57%
网络 (16-16 / 64-32 / 256-128)	16-16 / 64-32 / 256-128	10	0.01	94.28 / 96.92 / 97.77%
量化 (9 / 6 / 4 / 3 / 2 位)	128-64	—	—	97.53 / 97.53 / 97.53 / 97.53 / 97.37%
组合 (A / B / C / bad)	256-128 / 128-64	20/20/30/10	0.01 / 0.005 / 0.001 / 0.1	97.71 / 97.73 / 97.83 / 31.52%

注：除种子实验组外均取 `srand(50)`，`TRAIN_SIZE` 均取全量 60000（反面组合除外，为 1000），`WRONG_SHOW=0`。量化实验直接加载基线训练产出的全精度权重（128-64 结构）推理，因此 EPOCH 与 LR 不适用。括号内取值的顺序与“测试准确率”列的数值一一对应；各实验目录名依次为 `seed_1` / `seed_123` / `seed_2024`、`ep5` / `ep20` / `ep30`、`ts_1k` / `ts_5k` / `ts_10k` / `ts_30k`、`lr_0001` / `lr_0005` / `lr_005` / `lr_05` / `lr_1`、`net_16_16` / `net_64_32` / `net_256_128`、`quant_p9` / `p6` / `p4` / `p3` / `p2` / `p1`、`combo_best` / `combo_best2` / `combo_slow` / `combo_bad`。

参考文献

- [1] LeCun Y, Cortes C, Burges C J C. The MNIST database of handwritten digits[EB/OL]. <http://yann.lecun.com/exdb/mnist/>.
- [2] Goodfellow I, Bengio Y, Courville A. Deep Learning[M]. Cambridge: MIT Press, 2016.
- [3] LeCun Y, Bottou L, Bengio Y, et al. Gradient-based learning applied to document recognition[J]. Proceedings of the IEEE, 1998, 86(11): 2278–2324.